Exp No:11

Experiment to understand pandas library use cases in Data science.

AIM:

Understand data frame use cases using pandas library

PROGRAM:

```
import numpy as np
import pandas as pd
list=[[1,'Smith',50000],[2,'Jones',60000]]
df=pd.DataFrame(list)
df
df.head()
df.tail()
df.info()
df.salary
df.head()type(df.salary)
df.salary.mean()
df.salary.median()
df.salary.mode()
df.salary.var()
df.salary.std()
df.describe()
df.describe(include='all')
empCol=df.columns
empCol
emparray=df.values
emparray
employee_DF=pd.DataFrame(emparray,columns=empCol)
employee_DF
```

OUTPUT:

	0	1	2
0	1	Smith	50000
1	2	Jones	60000

	Empd	Name	Salary
0	1	Smith	50000
1	2	Jones	60000

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      object
 2   Salary  2 non-null      int64
dtypes: int64(2), object(1)
memory usage: 180.0+ bytes
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   R&D Spend        50 non-null     float64
 1   Administration   50 non-null     float64
 2   Marketing Spend  50 non-null     float64
 3   State            50 non-null     object
 4   Profit           50 non-null     float64
dtypes: float64(4), object(1)
memory usage: 2.1+ KB
```

	R&D Spend	Administration	Marketing Spend	State	Profit
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94

	R&D Spend	Administration	Marketing Spend	State	Profit

45	1000.23	124153.04	1903.93	New York	64926.08
46	1315.46	115816.21	297114.46	Florida	49490.75
47	0.00	135426.92	0.00	California	42559.73
48	542.05	51743.15	0.00	New York	35673.41
49	0.00	116983.80	45173.06	California	14681.40

	emp id	name	salary
0	1	SREE VARSSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH U G	8000

	emp id	name	salary
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH U G	8000
5	6	SRI HARSHAVARDHANAN R	3000
6	7	SRI HARSHAVARDHANAN R	6000

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp id  7 non-null      int64
 1   name    7 non-null      object
 2   salary  7 non-null      int64
dtypes: int64(2), object(1)
memory usage: 300.0+ bytes
```

	salary
0	5000
1	6000
2	7000
3	5000
4	8000
5	3000
6	6000

dtype: int64
pandas.core.series.Series
def __init__(data=None, index=None, dtype: Dtype | None=None, name=None, copy: bool | None=None, fastpath: bool | lib.NoDefault=lib.no_default) -> None
/usr/local/lib/python3.13/dist-packages/pandas/core/series.pyOne-dimensional ndarray with axis labels (including time series).

Labels need not be unique but must be a hashable type. The object supports both integer- and label-based indexing and provides a host of methods for performing operations involving the index. Statistical methods from ndarray have been overridden to automatically exclude missing data (currently represented as NaN).

Operations between Series (+, -, /, *, **) align values based on their associated index values-- they need not be the same length. The result index will be the sorted union of the two indexes.

Parameters
----
data : array-like, Iterable, dict, or scalar value
    Contains data stored in Series. If data is a dict, argument order is
    maintained.
index : array-like or Index (1d)
    Values must be hashable and have the same length as `data`.
    Non-unique index values are allowed. Will default to
    RangeIndex (0, 1, 2, ..., n) if not provided. If data is dict-like
    and index is None, then the keys in the data are used as the index. If the

    index is not None, the resulting Series is reindexed with the index values.
dtype : str, numpy.dtype, or ExtensionDtype, optional
    Data type for the output Series. If not specified, this will be
    inferred from `data`.
    See the :ref:`user guide <basics.dtypes>` for more usages.
name : Hashable, default None
    The name to give to the Series.
copy : bool, default False
    Copy input data. Only affects Series or 1d ndarray input. See examples.

Notes

Please reference the :ref:`User Guide <basics.series>` for more information.

Examples

Constructing Series from a dictionary with an Index specified

>>> d = {'a': 1, 'b': 2, 'c': 3}
>>> ser = pd.Series(data=d, index=['a', 'b', 'c'])
>>> ser
a   1
b   2
c   3
dtype: int64

The keys of the dictionary match with the Index values, hence the Index
values have no effect.

>>> d = {'a': 1, 'b': 2, 'c': 3}
>>> ser = pd.Series(data=d, index=['x', 'y', 'z'])
>>> ser
x   NaN
y   NaN
z   NaN
dtype: float64

Note that the Index is first build with the keys from the dictionary.
After this the Series is reindexed with the given Index values, hence we
get all NaN as a result.

Constructing Series from a list with `copy=False`.

>>> r = [1, 2]

```
>>> ser = pd.Series(r, copy=False)
>>> ser.iloc[0] = 999
>>> r
[1, 2]
>>> ser
0    999
1      2
dtype: int64
```

Due to input data type the Series has a `copy` of
the original data even though `copy=False`, so
the data is unchanged.

Constructing Series from a 1d ndarray with `copy=False`.

```
>>> r = np.array([1, 2])
>>> ser = pd.Series(r, copy=False)
>>> ser.iloc[0] = 999
>>> r
array([999,   2])
>>> ser
0    999
1      2
dtype: int64
```

Due to input data type the Series has a `view` on
the original data, so
the data is changed as well.

np.float64(5714.285714285715)
6000.0

	salary
0	5000
1	6000

dtype: int64
2571428.5714285714
1603.5674514745463

	emp id	salary
count	7.000000	7.000000
mean	4.000000	5714.285714
std	2.160247	1603.567451
min	1.000000	3000.000000
25%	2.500000	5000.000000
50%	4.000000	6000.000000
75%	5.500000	6500.000000
max	7.000000	8000.000000

	emp id	name	salary
count	7.000000	7	7.000000
unique	NaN	6	NaN
top	NaN	SRI HARSHAVARDHANAN R	NaN
freq	NaN	2	NaN
mean	4.000000	NaN	5714.285714
std	2.160247	NaN	1603.567451
min	1.000000	NaN	3000.000000
25%	2.500000	NaN	5000.000000
50%	4.000000	NaN	6000.000000
75%	5.500000	NaN	6500.000000
max	7.000000	NaN	8000.000000

```
Index(['emp id', 'name', 'salary'], dtype='object')
array([[1, 'SREE VARSSINI K S', 5000],
       [2, 'SREEMATHI B', 6000],
       [3, 'SREYA G', 7000],
       [4, 'SREYASKARI MULLAPUDI', 5000],
```

[5, 'SRI AKASH U G', 8000],
[6, 'SRI HARSHAVARDHANAN R', 3000],
[7, 'SRI HARSHAVARDHANAN R', 6000]], dtype=object)

	emp id	name	salary
0	1	SREE VARSSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH U G	8000
5	6	SRI HARSHAVARDHANAN R	3000
6	7	SRI HARSHAVARDHANAN R	6000

RESULT:

The experiment has been carried out successfully